

Name: Mokganyetjie Mahlatse Mphahlele

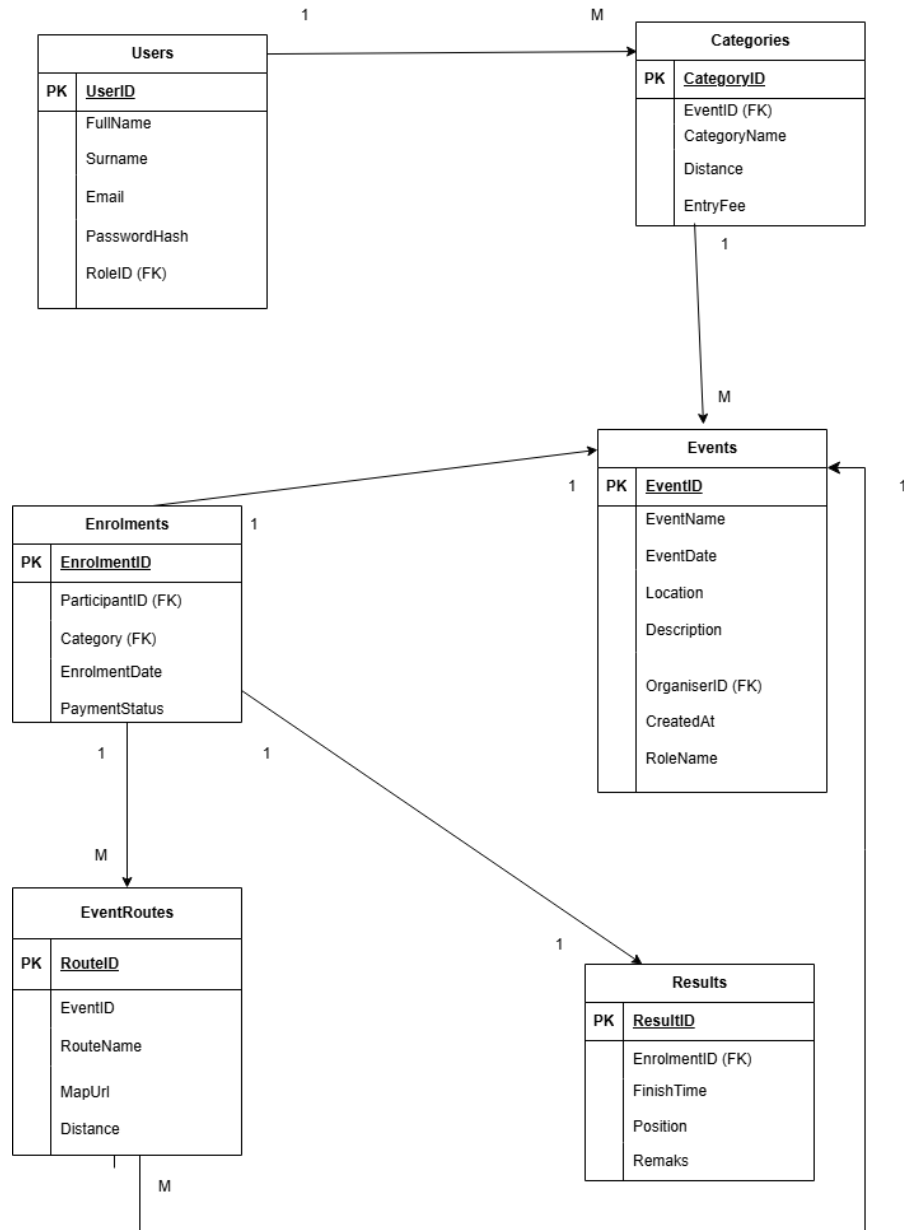
Student Number: 10481122

Module: Programming6212

Group 1

Assignment 1 (Part 1)

Section A



Section B

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
Post	/api/auth/register	Registers a new user account as either an Organiser or Participant.	None (Public)	{ firstName, lastName, email, password, role }	201 Created – User successfully registered. 400 Bad Request – Invalid or incomplete details. 409 Conflict – Email already registered.
Post	/api/auth/login	Authenticates a registered user and provides an authentication token.	None (Public)	{ email, password }	200 OK – Login successful and token returned. 401 Unauthorized – Invalid email or password.
Get	/api/users/profile	Retrieves the profile details of the currently logged-in user.	Any logged-in user	None	200 OK – User profile returned. 401 Unauthorized – Authentication required.

Put	/api/users/profile	Updates the profile details of the currently logged-in user.	Any logged-in user	{ firstName, lastName, email }	200 OK – Profile successfully updated. 400 Bad Request – Invalid information. 401 Unauthorized – Authentication required.
Get	/api/events	Retrieves all upcoming RaceDay events available on the platform.	None (Public)	None	200 OK – List of upcoming events.
Get	/api/events/{id}	Retrieves detailed information about a specific event.	None (Public)	None	200 OK – Event details returned. 404 Not Found – Event does not exist.
Post	/api/events	Creates a new running, walking or cycling event.	Organiser	{ name, description, eventDate, location }	201 Created – Event successfully created. 400 Bad Request – Invalid event information. 401 Unauthorized

					ed – Authentication required. 403 Forbidden – User is not an Organiser.
Put	/api/events/{id}	Updates the details of an existing event created by an Organiser.	Organiser	{ name, description, eventDate, location }	200 OK – Event successfully updated. 404 Not Found – Event does not exist. 403 Forbidden – User is not authorised.
Delete	/api/events/{id}	Deletes an existing event from the system.	Organiser	None	204 No Content – Event successfully deleted. 404 Not Found – Event does not exist. 403 Forbidden – User is not authorised.
Get	/api/events/{eventId}/categories	Retrieves all categories available for a	None (Public)	None	200 OK – List of event categories. 404 Not Found –

		specific event.			Event does not exist.
Post	/api/events/{eventId}/categories	Creates a new category for a specific event.	Organiser	{ name, distance, entryFee }	201 Created – Category successfully created. 400 Bad Request – Invalid category information . 404 Not Found – Event does not exist.
Put	/api/categories/{id}	Updates the details of an existing event category.	Organiser	{ name, distance, entryFee }	200 OK – Category successfully updated. 400 Bad Request – Invalid information . 404 Not Found – Category does not exist.
Delete	/api/categories/{id}	Deletes an existing event category.	Organiser	None	204 No Content – Category successfully deleted. 404 Not Found – Category does not exist. 403 Forbidden – User is

					not authorised.
Post	/api/events/{eventId}/enrolments	Enrols the logged-in Participant into a selected category for an event.	Participant	{ categoryId }	201 Created – Enrolment successfully created. 400 Bad Request – Invalid category. 404 Not Found – Event or category does not exist. 409 Conflict – Participant is already enrolled.
Get	/api/enrolments/my	Retrieves all event enrolments belonging to the logged-in Participant.	Participant	None	200 OK – Participant's enrolments returned. 401 Unauthorized – Authentication required.
Get	/api/events/{eventId}/enrolments	Retrieves all participant enrolments for a specific event.	Organiser	None	200 OK – List of event enrolments . 404 Not Found – Event does not exist. 403 Forbidden

					– User is not an Organiser.
Delete	/api/enrolments/{id}	Cancels an existing event enrolment belonging to the logged-in Participant.	Participant	None	204 No Content – Enrolment successfully cancelled. 404 Not Found – Enrolment does not exist. 403 Forbidden – User is not authorised.
Post	/api/events/{eventId}/results	Records a Participant's result after completing an event.	Organiser	{ enrolmentId, finishTime, position }	201 Created – Result successfully recorded. 400 Bad Request – Invalid result information. 404 Not Found – Enrolment does not exist.
Get	/api/results/my	Retrieves the logged-in Participant's personal race results and performance history.	Participant	None	200 OK – Participant's results returned. 401 Unauthorized – Authentication failed.

					ion required.
Get	/api/events/{eventId}/results	Retrieves the results of all participants who completed a specific event.	Organiser	None	200 OK – Event results returned. 404 Not Found – Event does not exist. 403 Forbidden – User is not an Organiser.
Put	/api/results/{id}	Updates an existing participant result.	Organiser	{ finishTime, position }	200 OK – Result successfully updated. 400 Bad Request – Invalid result information. 404 Not Found – Result does not exist.
Delete	/api/results/{id}	Deletes an incorrect or invalid participant result.	Organiser	None	204 No Content – Result successfully deleted. 404 Not Found – Result does not exist. 403 Forbidden – User is not authorised.

Section C

```
/* =====  
RACE DAY DATABASE  
Section C - SQL Database Script  
===== */
```

```
/* =====  
1. DATABASE CREATION  
===== */
```

```
IF DB_ID('RaceDayDB') IS NULL  
BEGIN  
    CREATE DATABASE RaceDayDB;  
END;  
GO
```

```
USE RaceDayDB;  
GO
```

```
/* =====  
2. DROP EXISTING TABLES  
Allows the script to be safely re-run.  
Tables are dropped in dependency order.  
===== */
```

```
IF OBJECT_ID('dbo.Results', 'U') IS NOT NULL
```

```
DROP TABLE dbo.Results;
```

```
IF OBJECT_ID('dbo.Enrolments', 'U') IS NOT NULL  
    DROP TABLE dbo.Enrolments;
```

```
IF OBJECT_ID('dbo.Categories', 'U') IS NOT NULL  
    DROP TABLE dbo.Categories;
```

```
IF OBJECT_ID('dbo.Events', 'U') IS NOT NULL  
    DROP TABLE dbo.Events;
```

```
IF OBJECT_ID('dbo.Users', 'U') IS NOT NULL  
    DROP TABLE dbo.Users;
```

```
IF OBJECT_ID('dbo.Roles', 'U') IS NOT NULL  
    DROP TABLE dbo.Roles;
```

```
GO
```

```
/* =====
```

3. CREATE TABLES

```
===== */
```

```
/* -----
```

Table 1: Roles

Stores the two roles supported by RaceDay:

Organiser and Participant.

----- */

```
CREATE TABLE dbo.Roles
(
    RoleId INT IDENTITY(1,1) PRIMARY KEY,
    RoleName NVARCHAR(50) NOT NULL UNIQUE
);
GO
```

/* -----

Table 2: Users

Stores both Organisers and Participants.

RoleId identifies the user's role.

----- */

```
CREATE TABLE dbo.Users
(
    UserId INT IDENTITY(1,1) PRIMARY KEY,
    FullName NVARCHAR(100) NOT NULL,
    Email NVARCHAR(100) NOT NULL UNIQUE,
    PasswordHash NVARCHAR(255) NOT NULL,
    RoleId INT NOT NULL,

    CONSTRAINT FK_Users_Roles
        FOREIGN KEY (RoleId)
        REFERENCES dbo.Roles(RoleId)
);
```

GO

/* -----

Table 3: Events

Stores RaceDay running, walking and cycling events.

OrganiserId identifies the organiser responsible
for the event.

----- */

CREATE TABLE dbo.Events

(

EventId INT IDENTITY(1,1) PRIMARY KEY,

EventName NVARCHAR(200) NOT NULL,

EventDate DATETIME NOT NULL,

Location NVARCHAR(200) NOT NULL,

Description NVARCHAR(MAX),

OrganiserId INT NOT NULL,

CreatedAt DATETIME NOT NULL DEFAULT GETDATE(),

CONSTRAINT FK_Events_Organiser

FOREIGN KEY (OrganiserId)

REFERENCES dbo.Users(UserId)

);

GO

/* -----

Table 4: Categories

Stores the different race categories available
for each event.

----- */

```
CREATE TABLE dbo.Categories
(
    CategoryId INT IDENTITY(1,1) PRIMARY KEY,
    EventId INT NOT NULL,
    CategoryName NVARCHAR(100) NOT NULL,
    Distance DECIMAL(6,2) NOT NULL,
    EntryFee DECIMAL(10,2) NOT NULL DEFAULT 0.00,

    CONSTRAINT FK_Categories_Events
        FOREIGN KEY (EventId)
        REFERENCES dbo.Events(EventId)
        ON DELETE CASCADE,

    CONSTRAINT CK_Categories_Distance
        CHECK (Distance > 0),

    CONSTRAINT CK_Categories_EntryFee
        CHECK (EntryFee >= 0)
);
GO
```

/* -----

Table 5: Enrolments

Connects Participants to event categories.

----- */

```
CREATE TABLE dbo.Enrolments
(
    EnrolmentId INT IDENTITY(1,1) PRIMARY KEY,
    ParticipantId INT NOT NULL,
    CategoryId INT NOT NULL,
    EnrolmentDate DATETIME NOT NULL DEFAULT GETDATE(),
    PaymentStatus NVARCHAR(20) NOT NULL DEFAULT 'Pending',

    CONSTRAINT FK_Enrolments_Participant
        FOREIGN KEY (ParticipantId)
        REFERENCES dbo.Users(UserId),

    CONSTRAINT FK_Enrolments_Category
        FOREIGN KEY (CategoryId)
        REFERENCES dbo.Categories(CategoryId),

    CONSTRAINT UQ_Enrolments_Participant_Category
        UNIQUE (ParticipantId, CategoryId),

    CONSTRAINT CK_Enrolments_PaymentStatus
        CHECK (PaymentStatus IN ('Pending', 'Paid', 'Cancelled'))
);
GO
```


/* -----

Table 6: Results

Stores the result achieved by a participant.

One enrolment can have one result.

----- */

CREATE TABLE dbo.Results

(

ResultId INT IDENTITY(1,1) PRIMARY KEY,

EnrolmentId INT NOT NULL UNIQUE,

FinishTime TIME NULL,

Position INT NULL,

Remarks NVARCHAR(255),

CONSTRAINT FK_Results_Enrolments

FOREIGN KEY (EnrolmentId)

REFERENCES dbo.Enrolments(EnrolmentId),

CONSTRAINT CK_Results_Position

CHECK (Position IS NULL OR Position > 0)

);

GO

/* =====

4. SEED DATA

===== */

```
/* -----  
Seed Roles  
----- */
```

```
INSERT INTO dbo.Roles (RoleName)  
VALUES  
('Organiser'),  
('Participant');  
GO
```

```
/* -----  
Seed Users  
2 Organisers  
2 Participants  
----- */
```

```
INSERT INTO dbo.Users  
(FullName, Email, PasswordHash, RoleId)  
VALUES  
('John Eventman', 'john@organiser.com', 'hashed_pw_1', 1),  
('Sarah Racepro', 'sarah@organiser.com', 'hashed_pw_2', 1),  
('Sipho Runner', 'sipho@participant.com', 'hashed_pw_3', 2),  
('Lerato Cyclist', 'lerato@participant.com', 'hashed_pw_4', 2);  
GO
```

/* -----

Seed Events

Three future RaceDay events.

----- */

INSERT INTO dbo.Events

(EventName, EventDate, Location, Description, OrganiserId)

VALUES

(

'Joburg City Run',

'2026-10-17 07:00:00',

'Johannesburg CBD',

'A scenic road running event through the heart of Johannesburg.',

1

),

(

'Durban Beach Walk',

'2026-11-07 08:30:00',

'Durban Promenade',

'A charity walking event along the Durban Golden Mile.',

1

),

(

'Cape Classic Cycle',

'2026-12-06 06:00:00',

'Cape Town',

'A cycling event featuring a challenging route through Cape Town.',

2

);

GO

/* -----

Seed Categories

Each event has one or more categories.

----- */

INSERT INTO dbo.Categories

(EventId, CategoryName, Distance, EntryFee)

VALUES

(1, '10km Elite Run', 10.00, 150.00),

(1, '5km Fun Run', 5.00, 80.00),

(2, '5km Promenade Walk', 5.00, 50.00),

(2, '10km Charity Walk', 10.00, 80.00),

(3, '100km Full Course', 100.00, 450.00),

(3, '50km Half Course', 50.00, 300.00);

GO

/* -----

Seed Enrolments

Participants are enrolled into event categories.

----- */

INSERT INTO dbo.Enrolments

(ParticipantId, CategoryId, PaymentStatus)

VALUES

(3, 1, 'Paid'),
(4, 5, 'Paid'),
(3, 3, 'Pending'),
(4, 2, 'Paid');

GO

/* -----

Seed Results

Sample result for an enrolled participant.

----- */

INSERT INTO dbo.Results

(EnrolmentId, FinishTime, Position, Remarks)

VALUES

(1, '00:34:12', 5, 'Personal Best');

GO

/* =====

5. VERIFICATION QUERIES

These can be used in SSMS to confirm that the
database was created and populated successfully.

===== */

SELECT * FROM dbo.Roles;

```
SELECT * FROM dbo.Users;  
SELECT * FROM dbo.Events;  
SELECT * FROM dbo.Categories;  
SELECT * FROM dbo.Enrolments;  
SELECT * FROM dbo.Results;  
GO
```

Commits

1. Initialise RaceDay project repository

2. Add README with RaceDay project overview
3. Create docs folder for Part 1 planning documents
4. Add initial RaceDay database entities
5. Define Users entity with Organiser and Participant roles
6. Define Events entity and organiser relationship
7. Define Categories entity and event relationship
8. Define Enrolments entity for participant event entries
9. Define Results entity and enrolment relationship
10. Add EventRoutes entity for event route information
11. Add primary keys and foreign keys to ERD
12. Add cardinality and relationship notation to ERD
13. Review and correct ERD relationships
14. Add final RaceDay ERD to docs folder
15. Create RaceDay API endpoint plan
16. Add authentication endpoints to API plan
17. Add user profile endpoints to API plan
18. Add event and category endpoints to API plan
19. Review API endpoint roles and expected responses